Name: Pranav Khera

Student Number: 24032995

Part I

High-frequency finance (HFF) relies on vast amounts of rapidly changing data, requiring databases designed for speed and scalability. SQL databases offer structured, relational storage, ensuring consistency but struggling with high-speed processing. NoSQL databases, with flexible schemas and distributed architecture, excel at handling voluminous, low-latency financial data.

SQL databases enforce a rigid relational structure, making them ideal for traditional financial records but less suited for irregularly spaced, high-frequency tick data. The limit order book (LOB) requires dynamic updates, which SQL's predefined schema may struggle to accommodate efficiently.

NoSQL databases can handle unstructured tick data, order book reconstructions, and evolving market conditions through schema-flexible structures. NoSQL's document-based or key-value models can store less structured data, useful for low-latency execution algorithms (e.g. TWAP, VWAP, percentage of volume) and market microstructure analysis. NoSQL's flexible schema enables efficient storage of granular level-2 tick data, reducing the need for preprocessing which improves execution latency.

SQL databases prioritise data integrity by following the ACID properties:

- Atomicity: Transactions are fully complete or fail (e.g. order execution must be all-or-nothing).
- Consistency: The database must be consistent before and after the transaction.
- Isolation: Transactions don't interfere with one another (e.g. concurrent trades are processed separately).
- Durability: Confirmed transactions occur even after system failures.

NoSQL databases instead follow CAP theorem, prioritising:

- Consistency: All nodes return the latest confirmed data to prevent discrepancies (e.g. Trade execution prices remain identical across all market data feeds).
- Availability: Every request gets a response, even if some nodes are down, though data may be slightly outdated (e.g. A trading system still executes orders during partial exchange outages).
- Partition Tolerance: The system remains operational despite network failures between nodes (e.g. A fragmented network still processes limit order book updates across available servers).

ACID ensures greater transaction security but can be slow for real-time bid-ask updates, trade execution, and order book modifications. NoSQL's availability-first model enables low-latency trade execution, important for HFF.

SQL databases scale vertically, requiring more CPU and RAM to handle increased workloads. This approach is costly and limited, making SQL less efficient for HFF, where large numbers of trades per second require fast processing. Furthermore, SQL queries depend on disk-based storage, introducing more latency.

NoSQL databases scale horizontally, distributing data across multiple servers, ensuring better availability and fault tolerance. This architecture is important for order book updates, price discovery mechanisms, and execution algorithms, requiring quick response times. Additionally, in-memory NoSQL databases reduce disk-based bottlenecks (e.g. in order routing and trade matching).

For storing and processing high-frequency finance data, speed, scalability, and flexibility are crucial, making NoSQL databases preferable to SQL. Their schema-less, distributed design allows trade execution, order book updates, and large-scale market data processing quickly. However, SQL remains important for structured financial records where data consistency and regulatory compliance take precedence. NoSQL is better suited for storing and processing HFF data due to its low-latency, high-speed architecture, while SQL is preferable for static financial data requiring strong integrity.

References:

GeeksforGeeks (2024) The Cap Theorem in DBMS, GeeksforGeeks. Available at: https://www.geeksforgeeks.org/the-cap-theorem-in-dbms/ (Accessed: 01 March 2025).

Anderson, B. and Nicholson, B. (2024) SQL vs. NoSQL databases: What's the difference?, IBM. Available at: https://www.ibm.com/think/topics/sql-vs-nosql#:~:text=SQL%20and%20NoSQL%20differ%20in,row%20transactions%20or%20unst (Accessed: 03 March 2025).

info, M. et al. (2025) Difference between SQL and NoSQL, GeeksforGeeks. Available at: https://www.geeksforgeeks.org/difference-between-sql-and-nosql/ (Accessed: 03 March 2025).

Part II

1) First, we unzip the compressed data file and the three CSV files, inside our terminal:

```
(base) MacBook-Air-273:~ Kherafamily$ cd documents/kcl/hff/hffcoursework1
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ unzip Data-20250220.zip
Archive:  Data-20250220.zip
  inflating: allGlaxoOrderDetail.CSV.gz
  inflating: allGlaxoOrderHistory.CSV.gz
  inflating: allGlaxoTradeReport.CSV.gz
  inflating: lse-data-description.pdf
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ gunzip allGlaxoOrderDetail.CSV.gz allGlaxoOrderHistory.CSV
.gz allGlaxoTradeReport.CSV.gz
```

2) Then we examine the first five rows (checking if there is a header row) and compare these to the formatting outlined in the 'lse-data-description.pdf' file. We also determine the total number of rows and columns in each dataset:

```
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ head -n 10 allGlaxoOrderDetail.CSV
709ENVUN07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1510.00000000,173,N,F,28022007,07:51:15,3717
208ATNHG07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1550.00000000,1800,N,F,18012007,15:48:21,654681
006D95WX07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1485.00000000,700,N,F,28022007,07:52:53,4338
006D94UH07,SET1,FE10,GB0009252882,GB,GBX,NULL,B,O,LO,1300.00000000,230,N,F,28022007,07:51:31,3849
709FJNIR07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1449.00000000,1100,N,F,28022007,16:30:54,1309558
709EPA1T07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1438.00000000,20000,N,F,28022007,16:30:18,1308066
709EODFR07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1464.00000000,1100,N,F,28022007,16:30:58,1309687
609JJXF807,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1448.00000000,38600,N,F,28022007,16:30:49,1309383
5099MB5A07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1438.00000000,12867,N,F,28022007,16:30:30,1308572
308PVR0Q07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1570.00000000,55,N,F,21022007,07:50:39,2740
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ head -n 10 allGlaxoOrderHistory.CSV
208VSG5Q07,M,709JKPU707,500,709JKPUL07,GB0009252882,GB,GBX,SET1,0,S,LO,364284,08032007,11:44:09
609NJZ0107,M,308XOPF507,243,308XOPF807,GB0009252882,GB,GBX,SET1,0,S,LO,221138,08032007,10:05:18
308WLUQQ07,M,609MGG5Y07,1680,609MGG5Z07,GB0009252882,GB,GBX,SET1,0,S,LO,661284,06032007,14:38:17
208SG8CP07,M,509ABGBD07,3288,509ABGBI07,GB0009252882,GB,GBX,SET1,0,S,LO,1114862,01032007,15:28:20
609MYSWA07,M,208V4A1707,12148,208V4A1807,GB0009252882,GB,GBX,SET1,0,B,LO,359978,07032007,11:28:39
609R1IHO07,M,408LULF907,392,408LULFE07,GB0009252882,GB,GBX,SET1,0,B,LO,286944,15032007,10:27:03
006QWO5E07,D,NULL,0,NULL,GB0009252882,GB,GBX,SET1,3656,B,LO,915280,23032007,16:21:59
3095JP3907,D,NULL,0,NULL,GB0009252882,GB,GBX,SET1,5000,B,LO,595012,23032007,14:04:59
509JJ8KI07,D,NULL,0,NULL,GB0009252882,GB,GBX,SET1,8419,S,LO,820174,20032007,15:44:21
20938C5X07,D,NULL,0,NULL,GB0009252882,GB,GBX,SET1,375,B,LO,57596,23032007,08:39:10
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ head -n 10 allGlaxoTradeReport.CSV
1114866,GB0009252882,SET1,GB,GBX,509ABGBI07,1419.00000000,3288,01032007,15:28:20,E,AT,N,Y,N,01032007,15:28:20
438192,GB0009252882,SET1,GB,GBX,308U55BX07,1423.00000000,1645,01032007,11:46:34,E,AT,N,Y,N,01032007,11:46:34
1285577,GB0009252882,SET1,GB,GBX,308UIQWF07,1421.00000000,298,01032007,16:14:20,E,AT,N,Y,N,01032007,16:14:20
736925,GB0009252882,SET1,GB,GBX,208T0L6W07,1401.50000000,3,02032007,13:43:40,A,X ,N,Y,N,02032007,13:43:41
1137035,GB0009252882,SET1,GB,GBX,609K7U8F07,1417.77140000,5,01032007,15:34:15,A,VW,N,Y,N,01032007,15:34:16
900321,GB0009252882,SET1,GB,GBX,208SCS1R07,1420.00000000,606,01032007,14:26:24,E,AT,N,Y,N,01032007,14:26:24
601775,GB0009252882,SET1,GB,GBX,408FNO6107,1404.00000000,1500,02032007,12:38:42,E,AT,N,Y,N,02032007,12:38:42
355811,GB0009252882,SET1,GB,GBX,308U3NHE07,1422.00000000,350,01032007,10:58:52,E,AT,N,Y,N,01032007,10:58:52
664624,GB0009252882,SET1,GB,GBX,408FKOGA07,1401.00000000,60,02032007,13:11:48,E,AT,N,Y,N,02032007,13:11:48
1191567,GB0009252882,SET1,GB,GBX,509B692D07,1415.00000000,601,02032007,16:17:29,E,AT,N,Y,N,02032007,16:17:29
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ wc -l allGlaxoOrderDetail.CSV allGlaxoOrderHistory.CSV all
GlaxoTradeReport.CSV
  274322 allGlaxoOrderDetail.CSV
  322208 allGlaxoOrderHistory.CSV
  130136 allGlaxoTradeReport.CSV
  726666 total
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ awk -F',' '{print NF}' allGlaxoOrderDetail.CSV | sort -nu
17
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ awk -F',' '{print NF}' allGlaxoOrderHistory.CSV | sort -nu
15
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ awk -F',' '{print NF}' allGlaxoTradeReport.CSV | sort -nu
17
```

3) We count the total number of NULL values in each dataset. After confirming that only two datasets contain NULLs, we check their column-wise distribution to check if any columns consist entirely of NULL values. Column 7 in the order details file, which contains the Participant Code, consists entirely of NULL values, and will be removed later:

```
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ grep -c "NULL" allGlaxoOrderDetail.CSV allGlaxoOrderHistor
y.CSV allGlaxoTradeReport.CSV
allGlaxoOrderDetail.CSV:274322
allGlaxoOrderHistory.CSV:205857
allGlaxoTradeReport.CSV:0
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ grep -o "NULL" allGlaxoOrderDetail.CSV allGlaxoOrderHistor
y.CSV allGlaxoTradeReport.CSV | wc -l
  686038
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ grep -o "NULL" allGlaxoOrderDetail.CSV | wc -l
  274324
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ grep -o "NULL" allGlaxoOrderHistory.CSV | wc -l
  411714
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ awk -F',' '{for (i=1; i<=NF; i++) if ($i == "NULL") count[
i]++}
> END {for (i=1; i<=NF; i++) print "Column " i ": " (count[i]+0) " NULL values"}' allGlaxoOrderDetail.CSV
Column 1: 0 NULL values
Column 2: 0 NULL values
Column 3: 0 NULL values
Column 4: 0 NULL values
Column 5: 0 NULL values
Column 6: 0 NULL values
Column 7: 274322 NULL values
Column 8: 0 NULL values
Column 9: 0 NULL values
Column 10: 0 NULL values
Column 11: 0 NULL values
Column 12: 0 NULL values
Column 13: 0 NULL values
Column 14: 0 NULL values
Column 15: 0 NULL values
Column 16: 0 NULL values
Column 17: 0 NULL values
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ awk -F',' '{for (i=1; i<=NF; i++) if ($i == "NULL") count[
i]++}
END {for (i=1; i<=NF; i++) print "Column " i ": " (count[i]+0) " NULL values"}' allGlaxoOrderHistory.CSV
Column 1: 0 NULL values
Column 2: 0 NULL values
Column 3: 205856 NULL values
Column 4: 0 NULL values
Column 5: 205856 NULL values
Column 6: 0 NULL values
Column 7: 0 NULL values
Column 8: 0 NULL values
Column 9: 0 NULL values
Column 10: 0 NULL values
Column 11: 0 NULL values
Column 12: 0 NULL values
Column 13: 0 NULL values
Column 14: 0 NULL values
Column 15: 0 NULL values
```

4) We check for and confirm that there are no duplicate rows in any of the datasets:

```
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ sort allGlaxoOrderDetail.CSV | uniq -d | head -n 10
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ sort allGlaxoOrderHistory.CSV | uniq -d | head -n 10
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ sort allGlaxoTradeReport.CSV | uniq -d | head -n 10
```

5) We convert the date format to YYYY-MM-DD as specified in the LSE documentation. Additionally, we remove the Participant Code column from the order details data set as it contains only NULL values. We create new cleaned files, ensuring our original datasets remain preserved:

```
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ awk -F',' 'BEGIN{OFS=","} {$15 = substr($15, 5, 4) "-" substr($15, 3, 2) "-" substr($15, 1, 2); print}' allGlaxoOrderDetail.CSV > cleaned_allGlaxoOrderDetai
l.CSV
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ awk -F',' 'BEGIN{OFS=","} {$14 = substr($14, 5, 4) "-" substr($14, 3, 2) "-" substr($14, 1, 2); print}' allGlaxoOrderHistory.CSV > cleaned_allGlaxoOrderHist
ory.CSV
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ awk -F',' 'BEGIN{OFS=","} {$9 = substr($9, 5, 4) "-" substr($9, 3, 2) "-" substr($9, 1, 2); print}' allGlaxoTradeReport.CSV > cleaned_allGlaxoTradeReport.CS
V
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ head -n 5 cleaned_allGlaxoOrderDetail.CSV cleaned_allGlaxoOrderHistory.CSV cleaned_allGlaxoTradeReport.CSV
==> cleaned_allGlaxoOrderDetail.CSV <==
709ENVUN07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1510.00000000,173,N,F,2007-02-28,07:51:15,3717
208ATNHG07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1550.00000000,1800,N,F,2007-01-18,15:48:21,654681
006D95WX07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1485.00000000,700,N,F,2007-02-28,07:52:53,4338
006D94UH07,SET1,FE10,GB0009252882,GB,GBX,NULL,B,O,LO,1300.00000000,230,N,F,2007-02-28,07:51:31,3849
709FJNIR07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1449.00000000,1100,N,F,2007-02-28,16:30:54,1309558

==> cleaned_allGlaxoOrderHistory.CSV <==
208VSG5Q07,M,709JKPU707,500,709JKPUL07,GB0009252882,GB,GBX,SET1,0,S,LO,364284,2007-03-08,11:44:09
609NJZ0107,M,308XQPF507,243,308XQPF807,GB0009252882,GB,GBX,SET1,0,S,LO,221138,2007-03-08,10:05:18
308WLUQQ07,M,609MGQ5Y07,1680,609MGQ5Z07,GB0009252882,GB,GBX,SET1,0,S,LO,661284,2007-03-06,14:38:17
208SG8CP07,M,509ABGBD07,3288,509ABGBI07,GB0009252882,GB,GBX,SET1,0,S,LO,1114862,2007-03-01,15:28:20
609MYSWA07,M,208V4A1707,12148,208V4A1807,GB0009252882,GB,GBX,SET1,0,B,LO,359978,2007-03-07,11:28:39

==> cleaned_allGlaxoTradeReport.CSV <==
1114866,GB0009252882,SET1,GB,GBX,509ABGBI07,1419.00000000,3288,2007-03-01,15:28:20,E,AT,N,Y,N,01032007,15:28:20
438192,GB0009252882,SET1,GB,GBX,308U55BX07,1423.00000000,1645,2007-03-01,11:46:34,E,AT,N,Y,N,01032007,11:46:34
1285577,GB0009252882,SET1,GB,GBX,308UIQWF07,1421.00000000,298,2007-03-01,16:14:20,E,AT,N,Y,N,01032007,16:14:20
736925,GB0009252882,SET1,GB,GBX,208T0LAW07,1401.50000000,3,2007-03-02,13:43:40,A,X ,N,Y,N,02032007,13:43:41
1137035,GB0009252882,SET1,GB,GBX,609K7U8F07,1417.77140000,5,2007-03-01,15:34:15,A,VW,N,Y,N,01032007,15:34:16
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ awk -F',' 'BEGIN{OFS=","} {for (i=1; i<=NF; i++) if (i!=7) printf "%s%s", $i, (i==NF?"\n":",")}' cleaned_allGlaxoOrderDetail.CSV > final_allGlaxoOrderDetail
.CSV
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ head -n 5 cleaned_allGlaxoOrderDetail.CSV
709ENVUN07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1510.00000000,173,N,F,2007-02-28,07:51:15,3717
208ATNHG07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1550.00000000,1800,N,F,2007-01-18,15:48:21,654681
006D95WX07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1485.00000000,700,N,F,2007-02-28,07:52:53,4338
006D94UH07,SET1,FE10,GB0009252882,GB,GBX,NULL,B,O,LO,1300.00000000,230,N,F,2007-02-28,07:51:31,3849
709FJNIR07,SET1,FE10,GB0009252882,GB,GBX,NULL,S,O,LO,1449.00000000,1100,N,F,2007-02-28,16:30:54,1309558
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ awk -F',' 'BEGIN{OFS=","} {for (i=1; i<=NF; i++) if (i!=7) printf "%s%s", $i, (i==NF?"\n":",")}' cleaned_allGlaxoOrderDetail.CSV > temp_file && mv temp_file
 cleaned_allGlaxoOrderDetail.CSV
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ head -n 5 cleaned_allGlaxoOrderDetail.CSV
709ENVUN07,SET1,FE10,GB0009252882,GB,GBX,S,O,LO,1510.00000000,173,N,F,2007-02-28,07:51:15,3717
208ATNHG07,SET1,FE10,GB0009252882,GB,GBX,S,O,LO,1550.00000000,1800,N,F,2007-01-18,15:48:21,654681
006D95WX07,SET1,FE10,GB0009252882,GB,GBX,S,O,LO,1485.00000000,700,N,F,2007-02-28,07:52:53,4338
006D94UH07,SET1,FE10,GB0009252882,GB,GBX,B,O,LO,1300.00000000,230,N,F,2007-02-28,07:51:31,3849
709FJNIR07,SET1,FE10,GB0009252882,GB,GBX,S,O,LO,1449.00000000,1100,N,F,2007-02-28,16:30:54,1309558
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ awk -F',' 'BEGIN{OFS=","} {$16 = substr($16, 5, 4) "-" substr($16, 3, 2) "-" substr($16, 1, 2); print}' cleaned_allGlaxoTradeReport.CSV > correctly_fixed_al
lGlaxoTradeReport.CSV
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ head -n 5 correctly_fixed_allGlaxoTradeReport.CSV
1114866,GB0009252882,SET1,GB,GBX,509ABGBI07,1419.00000000,3288,2007-03-01,15:28:20,E,AT,N,Y,N,2007-03-01,15:28:20
438192,GB0009252882,SET1,GB,GBX,308U55BX07,1423.00000000,1645,2007-03-01,11:46:34,E,AT,N,Y,N,2007-03-01,11:46:34
1285577,GB0009252882,SET1,GB,GBX,308UIQWF07,1421.00000000,298,2007-03-01,16:14:20,E,AT,N,Y,N,2007-03-01,16:14:20
736925,GB0009252882,SET1,GB,GBX,208T0LAW07,1401.50000000,3,2007-03-02,13:43:40,A,X ,N,Y,N,2007-03-02,13:43:41
1137035,GB0009252882,SET1,GB,GBX,609K7U8F07,1417.77140000,5,2007-03-01,15:34:15,A,VW,N,Y,N,2007-03-01,15:34:16
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ mv correctly_fixed_allGlaxoTradeReport.CSV cleaned_allGlaxoTradeReport.CSV
```

6) Attempting to import the data into MySQL results in an error. We inspect the datasets for hidden characters and identify extra characters at the end of the date column for all 3 datasets. These are removed to ensure compatibility with MySQL:

```
==> cleaned_allGlaxoOrderDetail.CSV <==
709ENVUN07,SET1,FE10,GB0009252882,GB,GBX,S,O,LO,1510.00000000,173,N,F,2007-02-28,07:51:15,3717^M
208ATNHG07,SET1,FE10,GB0009252882,GB,GBX,S,O,LO,1550.00000000,1800,N,F,2007-01-18,15:48:21,654681^M
006D95WX07,SET1,FE10,GB0009252882,GB,GBX,S,O,LO,1485.00000000,700,N,F,2007-02-28,07:52:53,4338^M
006D94UH07,SET1,FE10,GB0009252882,GB,GBX,B,O,LO,1300.00000000,230,N,F,2007-02-28,07:51:31,3849^M
709FJNIR07,SET1,FE10,GB0009252882,GB,GBX,S,O,LO,1449.00000000,1100,N,F,2007-02-28,16:30:54,1309558^M

==> cleaned_allGlaxoOrderHistory.CSV <==
208VSG5QO7,M,709JKPU707,500,709JKPUL07,GB0009252882,GB,GBX,SET1,0,S,LO,364284,2007-03-08,11:44:09^M
609NJZ0107,M,308XOPF507,243,308XOPF807,GB0009252882,GB,GBX,SET1,0,S,LO,221138,2007-03-08,10:05:18^M
308WLUQQ07,M,609MGG5Y07,1680,609MGG5Z07,GB0009252882,GB,GBX,SET1,0,S,LO,661284,2007-03-06,14:38:17^M
208SQ8CP07,M,509ABGBD07,3288,509ABGBI07,GB0009252882,GB,GBX,SET1,0,S,LO,1114862,2007-03-01,15:28:20^M
609MYSWA07,M,208V4A1707,12148,208V4A1807,GB0009252882,GB,GBX,SET1,0,B,LO,359978,2007-03-07,11:28:39^M

==> cleaned_allGlaxoTradeReport.CSV <==
1114866,GB0009252882,SET1,GB,GBX,509ABGBI07,1419.00000000,3288,2007-03-01,15:28:20,E,AT,N,Y,N,2007-03-01,15:28:20^M
438192,GB0009252882,SET1,GB,GBX,308U55BX07,1423.00000000,1645,2007-03-01,11:46:34,E,AT,N,Y,N,2007-03-01,11:46:34^M
1285577,GB0009252882,SET1,GB,GBX,308UIQWF07,1421.00000000,298,2007-03-01,16:14:20,E,AT,N,Y,N,2007-03-01,16:14:20^M
736925,GB0009252882,SET1,GB,GBX,208T0L6W07,1401.50000000,3,2007-03-02,13:43:40,A,X ,N,Y,N,2007-03-02,13:43:41^M
1137035,GB0009252882,SET1,GB,GBX,609K7U8F07,1417.77140000,5,2007-03-01,15:34:15,A,VW,N,Y,N,2007-03-01,15:34:16^M
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ sed -i '' 's/\r//g' cleaned_allGlaxoOrderDetail.CSV
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ sed -i '' 's/\r//g' cleaned_allGlaxoOrderHistory.CSV
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ sed -i '' 's/\r//g' cleaned_allGlaxoTradeReport.CSV
(base) MacBook-Air-273:hffcoursework1 Kherafamily$ head -n 5 cleaned_allGlaxoOrderDetail.CSV cleaned_allGlaxoOrderHistory.CSV cleaned_allGlaxoTradeReport.CSV | cat -v
==> cleaned_allGlaxoOrderDetail.CSV <==
709ENVUN07,SET1,FE10,GB0009252882,GB,GBX,S,O,LO,1510.00000000,173,N,F,2007-02-28,07:51:15,3717
208ATNHG07,SET1,FE10,GB0009252882,GB,GBX,S,O,LO,1550.00000000,1800,N,F,2007-01-18,15:48:21,654681
006D95WX07,SET1,FE10,GB0009252882,GB,GBX,S,O,LO,1485.00000000,700,N,F,2007-02-28,07:52:53,4338
006D94UH07,SET1,FE10,GB0009252882,GB,GBX,B,O,LO,1300.00000000,230,N,F,2007-02-28,07:51:31,3849
709FJNIR07,SET1,FE10,GB0009252882,GB,GBX,S,O,LO,1449.00000000,1100,N,F,2007-02-28,16:30:54,1309558

==> cleaned_allGlaxoOrderHistory.CSV <==
208VSG5QO7,M,709JKPU707,500,709JKPUL07,GB0009252882,GB,GBX,SET1,0,S,LO,364284,2007-03-08,11:44:09
609NJZ0107,M,308XOPF507,243,308XOPF807,GB0009252882,GB,GBX,SET1,0,S,LO,221138,2007-03-08,10:05:18
308WLUQQ07,M,609MGG5Y07,1680,609MGG5Z07,GB0009252882,GB,GBX,SET1,0,S,LO,661284,2007-03-06,14:38:17
208SQ8CP07,M,509ABGBD07,3288,509ABGBI07,GB0009252882,GB,GBX,SET1,0,S,LO,1114862,2007-03-01,15:28:20
609MYSWA07,M,208V4A1707,12148,208V4A1807,GB0009252882,GB,GBX,SET1,0,B,LO,359978,2007-03-07,11:28:39

==> cleaned_allGlaxoTradeReport.CSV <==
1114866,GB0009252882,SET1,GB,GBX,509ABGBI07,1419.00000000,3288,2007-03-01,15:28:20,E,AT,N,Y,N,2007-03-01,15:28:20
438192,GB0009252882,SET1,GB,GBX,308U55BX07,1423.00000000,1645,2007-03-01,11:46:34,E,AT,N,Y,N,2007-03-01,11:46:34
1285577,GB0009252882,SET1,GB,GBX,308UIQWF07,1421.00000000,298,2007-03-01,16:14:20,E,AT,N,Y,N,2007-03-01,16:14:20
736925,GB0009252882,SET1,GB,GBX,208T0L6W07,1401.50000000,3,2007-03-02,13:43:40,A,X ,N,Y,N,2007-03-02,13:43:41
1137035,GB0009252882,SET1,GB,GBX,609K7U8F07,1417.77140000,5,2007-03-01,15:34:15,A,VW,N,Y,N,2007-03-01,15:34:16
```

7) We start MySQL, ensuring that local_infile is set to 1 meaning we can import locally stored files into the relevant database:

```
(base) MacBook-Air-273:~ Kherafamily$ mysql -u root -p --local-infile=1
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 28
Server version: 9.2.0 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> set global local_infile=1;
Query OK, 0 rows affected (0.00 sec)

mysql> show variables like 'local_infile';
+---------------+-------+
| Variable_name | Value |
+---------------+-------+
| local_infile  | ON    |
+---------------+-------+
1 row in set (0.04 sec)
```

8) We create a MySQL database, 'glaxo_orders', where we will store the cleaned datasets:

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| K24032995          |
| lse_high_freq      |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
6 rows in set (0.06 sec)

mysql> CREATE DATABASE glaxo_orders;
Query OK, 1 row affected (0.06 sec)

mysql> use glaxo_orders;
Database changed
```

9) Within our new database 'glaxo_orders', we create three separate tables for our cleaned datasets and check that they have been stored in the correct location. Since the datasets contain multiple shared attributes (currency code, market mechanism type

etc.), we can either define composite primary keys consisting of multiple columns or avoid using primary keys. For the purposes of this task, we avoid using any primary keys:

```
mysql> CREATE TABLE order_details (
    -> OrderCode VARCHAR(20),
    -> MarketSegmentCode VARCHAR(10),
    -> MarketSectorCode VARCHAR(10),
    -> TICode VARCHAR(20),
    -> CountryOfRegister VARCHAR(5),
    -> CurrencyCode VARCHAR(5),
    -> BuySellInd CHAR(1),
    -> MarketMechanismGroup VARCHAR(10),
    -> MarketMechanismType VARCHAR(10),
    -> Price DECIMAL(18,8),
    -> AggregateSize INT,
    -> SingleFillInd CHAR(1),
    -> BroadcastUpdateAction CHAR(1),
    -> Date DATE,
    -> Time TIME,
    -> MessageSequenceNumber INT
    -> );
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> CREATE TABLE order_history (
    -> OrderCode VARCHAR(20),
    -> OrderActionType CHAR(1),
    -> MatchingOrderCode VARCHAR(20) NULL,
    -> TradeSize INT,
    -> TradeCode VARCHAR(20) NULL,
    -> TICode VARCHAR(20),
    -> CurrencyCode VARCHAR(5),
    -> CountryOfRegister VARCHAR(5),
    -> MarketSegmentCode VARCHAR(10),
    -> AggregateSize INT,
    -> BuySellInd CHAR(1),
    -> MarketMechanismType VARCHAR(10),
    -> MessageSequenceNumber INT,
    -> Date DATE,
    -> Time TIME
    -> );
Query OK, 0 rows affected (0.02 sec)
```

```
mysql> CREATE TABLE trade_reports (
    -> MessageSequenceNumber INT,
    -> TICode VARCHAR(20),
    -> MarketSegmentCode VARCHAR(10),
    -> CountryOfRegister VARCHAR(5),
    -> CurrencyCode VARCHAR(5),
    -> TradeCode VARCHAR(20),
    -> TradePrice DECIMAL(18,8),
    -> TradeSize INT,
    -> TradeDate DATE,
    -> TradeTime TIME,
    -> BroadcastUpdateAction CHAR(1),
    -> TradeTypeInd VARCHAR(5),
    -> TradeTimeInd CHAR(1),
    -> BargainConditions CHAR(1),
    -> ConvertedPriceInd CHAR(1),
    -> PublicationDate DATE NULL,
    -> PublicationTime TIME
    -> );
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> show tables;
+------------------------+
| Tables_in_glaxo_orders |
+------------------------+
| order_details          |
| order_history          |
| trade_reports          |
+------------------------+
3 rows in set (0.02 sec)
```

10) We load the datasets into their respective MySQL tables:

```
mysql> LOAD DATA LOCAL INFILE '/Users/Kherafamily/Documents/KCL/HFF/HFFCoursework1/cleaned_allGlaxoOrderDetail.CSV'
    -> INTO TABLE order_details
    -> FIELDS TERMINATED BY ',';                                                    Query OK, 274322 rows affected (1.57 sec)
Records: 274322  Deleted: 0  Skipped: 0  Warnings: 0

mysql> LOAD DATA LOCAL INFILE '/Users/Kherafamily/Documents/KCL/HFF/HFFCoursework1/cleaned_allGlaxoOrderHistory.CSV'
    -> INTO TABLE order_history
    -> FIELDS TERMINATED BY ',';
Query OK, 322208 rows affected (1.44 sec)
Records: 322208  Deleted: 0  Skipped: 0  Warnings: 0

mysql> LOAD DATA LOCAL INFILE '/Users/Kherafamily/Documents/KCL/HFF/HFFCoursework1/cleaned_allGlaxoTradeReport.CSV'
    -> INTO TABLE trade_reports
    -> FIELDS TERMINATED BY ',';                                                    Query OK, 130136 rows affected (0.79 sec)
Records: 130136  Deleted: 0  Skipped: 0  Warnings: 0
```

11) To verify data integrity, we count the number of cancelled orders in both the original dataset and our MySQL database. As we can see below, there are 201,675 cancelled orders, matching the result obtained via a Python query in Part III:

```
(base) MacBook-Air-273:HFFCoursework1 Kherafamily$ awk -F',' '$2 == "D" {count++} END {print count}' a
llGlaxoOrderHistory.CSV
201675
```

```
mysql> SELECT COUNT(*) AS CancelledOrders FROM order_history WHERE OrderActionType = 'D';
+-----------------+
| CancelledOrders |
+-----------------+
|          201675 |
+-----------------+
1 row in set (0.38 sec)
```

Part III - Here we establish a connection to the MySQL database and perform some operations on it. Our first query counts the number of cancelled orders, which matches the result obtained from our original dataset and our cleaned dataset imported into MySQL.

In [19]:
```
import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="Vedant1711",
    database="glaxo_orders"
)

cursor = conn.cursor()

query = "SELECT COUNT(*) FROM order_history WHERE OrderActionType = 'D';"
cursor.execute(query)
cancelled_orders = cursor.fetchone()[0]

print(f"Total Cancelled Orders: {cancelled_orders}")


query2 = "SELECT MIN(TradeDate) FROM trade_reports;"

cursor.execute(query2)
earliest_trade = cursor.fetchone()[0]

query3 = "SELECT MAX(TradeDate) FROM trade_reports;"

cursor.execute(query3)
last_trade = cursor.fetchone()[0]

query4 = """
SELECT TradeDate, COUNT(*) AS TradeCount
FROM trade_reports
GROUP BY TradeDate
ORDER BY TradeCount DESC
LIMIT 5;
"""
cursor.execute(query4)
top_trade_days = cursor.fetchall()

query5 = """
SELECT BuySellInd, COUNT(*) AS OrderCount
FROM order_details
GROUP BY BuySellInd;
```

```
"""

cursor.execute(query5)
order_counts = cursor.fetchall()


for row in order_counts:
    print(f"Order Direction - {row[0]}, Count: {row[1]}")

query6 = """
SELECT MarketMechanismType, COUNT(*) AS OrderCount
FROM order_details
GROUP BY MarketMechanismType
ORDER BY OrderCount DESC;
"""

cursor.execute(query6)
order_type_counts = cursor.fetchall()

for row in order_type_counts:
    print(f"Order Type - {row[0]}, Count: {row[1]}")



print(f"Date of First Trade: {earliest_trade}")
print(f"Date of Last Trade: {last_trade}")
print("Top 5 Days with Most Trades:")
for row in top_trade_days:
    print(f"Date - {row[0]}, trades: {row[1]}")

cursor.close()
conn.close()
```

```
Total Cancelled Orders: 201675
Order Direction - S, Count: 133266
Order Direction - B, Count: 141056
Order Type - LO, Count: 272945
Order Type - MO, Count: 1377
Date of First Trade: 2007-02-27
Date of Last Trade: 2007-03-30
Top 5 Days with Most Trades:
Date - 2007-03-01, trades: 8167
Date - 2007-03-05, trades: 7427
Date - 2007-03-02, trades: 7083
Date - 2007-03-23, trades: 7060
Date - 2007-03-14, trades: 6714
```